

The prime wavelet tree: comparison of the stored files

All Carmichael numbers below 10^{24} : $N = 308,279,939$ (CNs). Every number on this page comes from `results/cells.tsv`.

	ABC	bits per CN	build (min)	CNs (min)	CNs + primes	ABC	bits per CN	build (min)	CNs (min)	CNs + primes
raw n	000	61.31	0.43	0.05	33.4	001	40.10	1.65	0.76	34.1
$d_{\min}$	100	31.67	158.0	214.5 [†]	223.4	101	24.82	159.1	215.1 [†]	224.0
PWT	010	67.24	7.75	0.30	0.30	011	32.03	12.7	2.81	2.81
PWT + $d_{\min}$	110	33.65	167.1	1.31	10.1	111	15.70	168.5	2.15	11.0
PWT + d^*						111 [‡]	15.28	2011 [§]	2.12	10.8

Switch A stores $d_{\min}(n)$ in place of n . Switch B stores a prime wavelet tree in place of a sorted list. Switch C applies every coding trick. With $C = 0$, each value or symbol is one VByte. Times are single-threaded minutes on one machine (page 2). Bold marks the best value in each column. Dark blue marks the smallest file. [†]The $d_{\min}$ list itself decodes in 0.03 and 0.61 minutes. The rest is factoring every d . [‡]The heuristic divisors d^* replace $d_{\min}$. [§]Wall-clock time as run. Extraction ran on 15 threads, the search on 6 and encoding on 4. The search alone took 119,102 s.

cell	file	bytes	bits/CN	× text	certify (s)	SHA-256 (prefix)	hosted
000	c000-n-vbyte.vbg	2,362,758,686	61.315	7.77	86.07	5778a94f	no
001	carmichael-1e24-cell1001-n-classes.cnd	1,545,310,979	40.101	11.88	130.67	0a623eae	yes
100	c100-dmin-vbyte.vbg	1,220,449,393	31.671	15.04	1594.12 (13 thr.)	57e5be32	no
101	carmichael-1e24-cell1101-dmin-classes.cnd	956,341,716	24.817	19.20	1347 (13 thr.)	d6607faa	yes
010	c010-pwt-full-bytes.pwt2n	2,591,083,919	67.240	7.09	100.5	b7e2e689	no
011	carmichael-1e24-cell1011-pwt-full.pwt2	1,234,242,404	32.029	14.87	246.6	dd69b283	yes
110	c110-pwt-dmin-bytes.pwt2n	1,296,532,881	33.646	14.16	156.7	d2ba4ade	no
111	carmichael-1e24-cell1111-pwt-dmin.pwt2	604,904,156	15.698	30.35	204.7	0b387083	yes
111 [‡]	carmichael-1e24-cell1111x-pwt-dstar.pwt2	588,684,305	15.277	31.19	203.0	3bc8369b	yes
d^* masks	c111x-dstar-masks.bin	616,559,886	16.000	29.78		e599e761	no
oracle	carmichael-1e24-oracle.orc1	4,462,931,093	115.815	4.11		a970d7d6	yes
text	new_table.txt	18,358,310,309	476.406	1.00		c934af2a	no
gzip -9	of the text	7,222,338,403	187.423	2.54			
zstd	of the text	6,065,397,406	157.400	3.03			
xz	of the text	5,882,849,796	152.663	3.12			

SHA256SUMS gives the full SHA-256 of each hosted file, and `cells/*/CELL` gives it for every file. The d^* masks are the checkpoint of the search. They are not hosted. The authors of the table serve the text.

The hosted files are at <https://blue.butler.edu/~jewebste/prime-wavelet-tree/v1/>.

Definitions and sources

Space. Space is the file size in bytes $\times 8/N$, with $N = 308,279,939$. The VByte list files of cells 000 and 100 include a 24-byte header.

Build. Build is the wall-clock time from the oracle to the closed file. It has two steps. An extraction step turns the oracle into the encoder's input, which is the list of n , the list of $d_{\min}$ or the prime paths. Then the encode step writes the file. Each step ran on one thread on an otherwise idle machine. The time is the median of three runs where three exist. For $A = 1$ almost all of it is the search for $d_{\min}$. On 15 threads that search takes 17.9 minutes to the list of $d_{\min}$ and 18.6 minutes to the $d_{\min}$ paths. The PWT + d^* build is quoted as run, with its threads in the footnote, because its divisor search ran once. The file stores the chosen divisors, so it can be rebuilt without repeating the search. That rebuild takes about 13 minutes from the file's own exported paths (`make rebuild-111x`).

Decode, CNs. This is a single-thread decode in memory to every CN. It excludes loading the file, checking its payload hash and sieving (2–10 s). The lists of n decode in sorted order. The trees decode in tree order, and sorting adds 0.8 minutes. The $d_{\min}$ lists yield only the divisors d . Each CN is $n = d(d^{-1} \bmod \lambda(d))$ once d is factored, at $41.75 \mu\text{s}$ per number.

Decode, CNs + primes. This is the full content of the text table, every CN with its prime factorization. The trees over full factorizations deliver every prime as they decode. The other files need factoring. Its cost per number was measured on one thread on 3,084,288 CNs (every 100th oracle block), and every factorization was checked against the oracle. Factoring n from scratch takes $6.495 \mu\text{s}$ (Shallue–Webster Algorithm 1). Factoring the cofactor $r^* = n/d$ takes $1.7185 \mu\text{s}$ for $d_{\min}$ and $1.6805 \mu\text{s}$ for d^* .

Certify. Certify decodes the file, sorts it, checks the footer targets and compares every CN with the oracle, in order. The time includes hashing the oracle file. The trees were certified on 2026-09-26 on an otherwise idle machine. Cell 101 was certified on 2026-08-19 on an otherwise idle machine. Cells 000, 001 and 100 were certified on 2026-10-06 while other jobs ran. Page 1 gives the thread counts.

Verified. On 2026-10-06 every hosted file was certified against the oracle. So were the rebuilt files of cells 000 and 100. Cells 010 and 110 were certified on 2026-09-26. On 2026-10-06 the encoders also rewrote cells 000, 001 and 100 byte for byte from the hash-checked lists of n and $d_{\min}$. They rewrote the four tree files from the hash-checked path files. They rewrote the d^* file twice, once from its own exported paths and once from the masks the search saved, which are not hosted. A clean copy of this repository, built from its sources, then verified every file of page 1 and the oracle (`make verify-CELL`). It rebuilt cells 001 and 101 and the d^* file, the last from the saved masks (`make rebuild-CELL`). A second clean copy rebuilt cells 000, 010, 100 and 110 from the oracle alone, every intermediate included. It also rebuilt the d^* file from its own exported paths. Every rebuild matched the published file byte for byte. The prefix self-test (`make check`) runs every encoder and decoder on the 8,241 CNs below 10^{12} against frozen hashes.

Machine. Every time was measured on an Intel Core i9-9900K (8 cores, 16 threads), 64 GB RAM, NVMe SSD, Windows 10. The compiler is GCC 16.1.0 (`w64devkit`), `-std=c++20 -O2 -march=native`. The tree encoder's model choice and the d^* search use floating point. The encoders' rebuilds reproduce the published bytes on an x86-64 build with FMA3 (`NATIVE=1`). The search ran once and has not been repeated. Every decoder is integer-only.

Data. The table of Carmichael numbers below 10^{24} is by A. Shallue and J. Webster (arXiv:2506.09903). Its text is at https://blue.butler.edu/~jewebsite/new_table.txt. It has 18,358,310,309 bytes and SHA-256

`c934af2a208832689586d12bbfe9e9cfc4ccb6447375baa39308d2e3373e6170`.

The oracle, `carmichael-1e24-oracle.orc1`, has 4,462,931,093 bytes and SHA-256

`a970d7d665e3ce6e11b5017371833a2ae9fafc12ef5509f0cba12ce102a5815f`.

It holds every n and its prime factors, each proven prime. It decodes to the text byte for byte.

Sources. The build times come from `results/runs/construction_*`. The two $d_{\min}$ times on 15 threads are runs X2O15b (stamp total) and X3 (median of three) in `results/runs/construction_round2_summary.txt` and `construction_round1_summary.txt`. The decode and factoring times come from `cells/*/runs/time*` and the factoring logs. The certify times come from `cells/*/runs/`. The rebuilds and certifications of 2026-10-06 are in `cells/*/runs/rebuild_*` and `certify_*`. The logs of the clean copies of this repository are `cells/*/runs/repo_*`, `results/runs/repo-validation-2026-10-06/` and `results/runs/review-2026-10-06/`. The bytes and hashes come from the files themselves (SHA256SUMS).